

Preface

The majority of this guide text, was generated using Gemini. I modified or replaced some of the explanations and added screenshots.

LM Studio allows you to run large language models (LLMs) locally on your PC or Mac entirely offline. Here is a quick guide to setting it up and serving models locally.

Installation

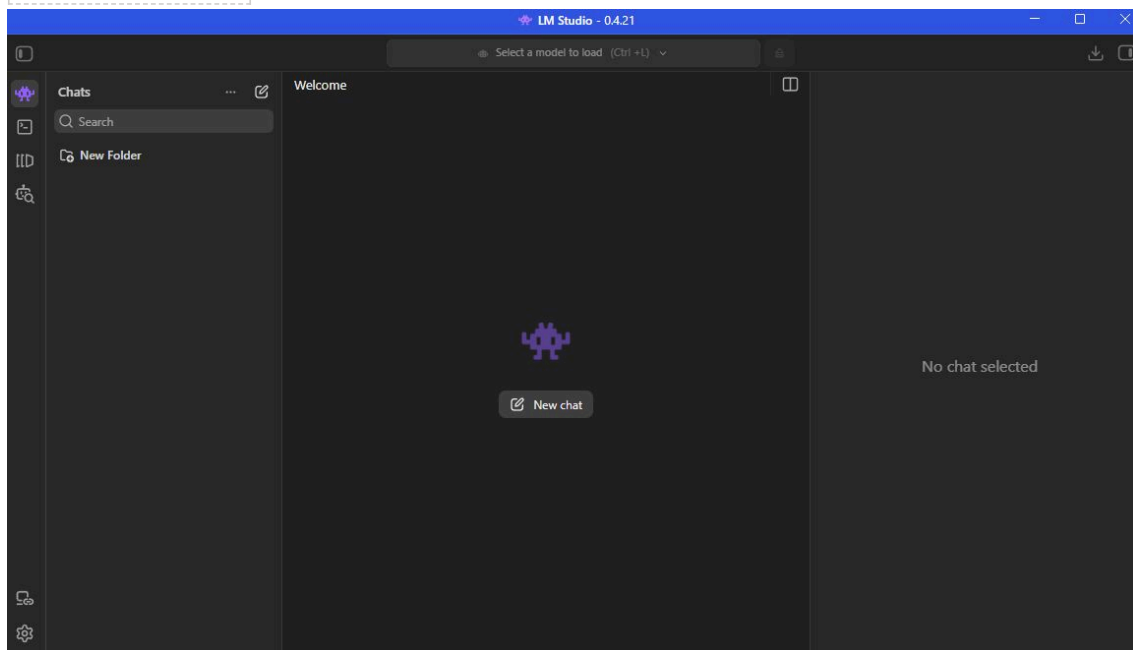
- Navigate to the official website: lmstudio.ai.
- Click **Download LM Studio** for your operating system (Windows, macOS, or Linux).
- Run the installer (.exe, .dmg, or .ApplImage) and complete the standard wizard steps. No manual dependencies or extra drivers are required.

Interface Overview

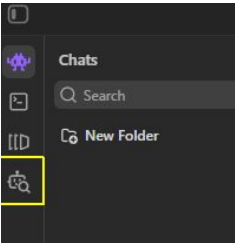
LM Studio uses a left sidebar for navigation across its primary modules:

- **Chat** (💬): An interactive, ChatGPT-style playground to chat directly with your downloaded models.
- **Developer / Local Server** (< />): The control hub to expose loaded models as an OpenAI-compatible local API server.
- **My Models** (📁): File manager for viewing, moving, or deleting local .gguf weights on disk.
- **Model Search / Discover** (🔍): A integrated search engine connected directly to Hugging Face GGUF repositories.

Remote image blocked



Downloading LLM Models



- Click the **Search icon** (🔍) on the left sidebar.
- Type the name of a model into the search bar (e.g., Qwen2.5-Coder, Llama-3.2, or Phi-4-mini).
- Select a repository from the search results. The right sidebar will display available quantization files (.gguf).
- Look for quantization marked **"Too large for your system"** versus **"Full GPU Acceleration"** / **"Fits in VRAM"**.
- Click **Download** next to your target quantization (e.g., Q4_K_M).

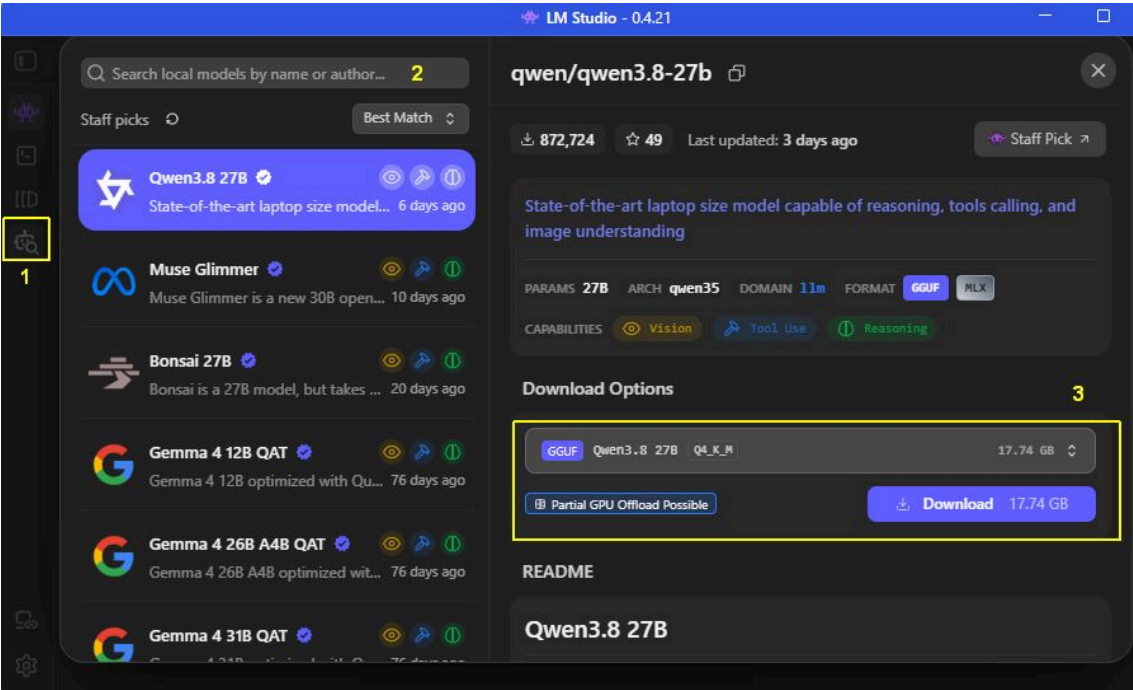
"Q8_0" quantized models should generally provide the best quality results, but that does not mean you need to use them, you could also use "Q4_K_M" ones.

The goal is to find a model that fits comfortably within your available memory while still providing good results. Smaller models are usually preferable for our use case.

I try to follow the principle of **"the smallest model that provides a plausible result."** I don't need the latest or largest model - just one that produces good enough mission descriptions.

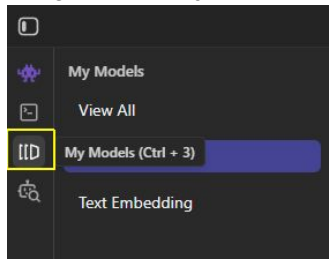
Models I tested and used:

Model Name	Size	Link	Comments
qwen2.5-coder-7b-instruct Q_8 or Q4_K_M	8.1G 4.68G	hugging_face	Probably one of the better models to use to construct description.
liquid/lfm2.5-1.2b	1.2G	LFM2.5 1.2B Instruct Q8_0	Generates basic mission description
qwen/qwen3-1.7b	1.7G	Qwen3 1.7B Q8_0	Generates basic mission description.



[Optional] Configure Model Parameters

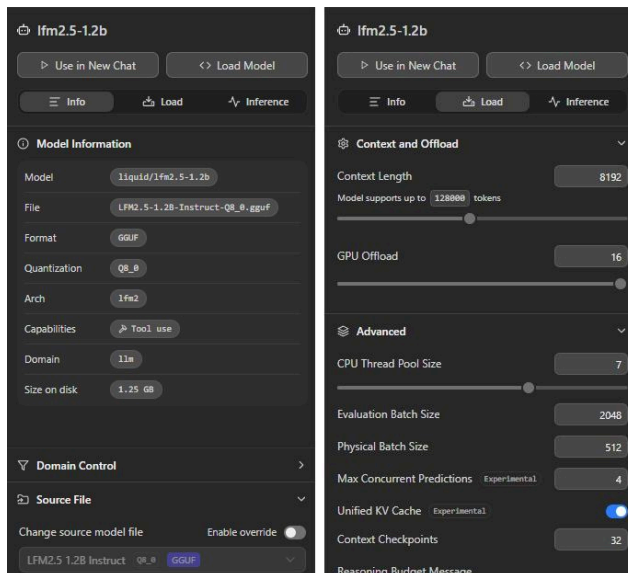
Navigate to the "My Models" screen.



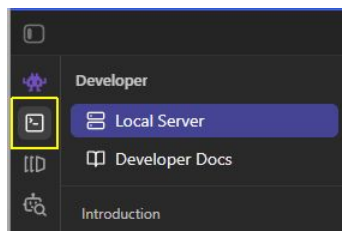
You will see the list of the download models. Pick one. On the right side you will see "info" and "load" tabs. The "load" tab will allow you to set up the defaults properties you would like to "start the model" (used later).

My Suggestion based on Mission-X needs are:

- Context Length: 1024 or 2048.
- Tokens: 1024 are enough.
- **GPU Offload:** Should be safe to use. Monitor how your simulator behaves while the LLM is running.
~~Zero ("0"). Disable GPU usage, we want 100% for X-Plane (But it is your decision).~~
- CPU Thread Pool Size: 4-6. We will let our CPU do the bulk of work.
- Offload KV cache to GPU Memory: **disable**. We want VRAM to be fully allocated by X-Plane and not the LLM.



Load and Expose as a Local Server (Developer Tab)



If you are using the LLM with other applications, I recommend starting the main application first and only then starting the LLM server.

The main reason is that LLM models and their servers can consume significant system resources. It is therefore better to start them only after the main application is already running and using the resources it needs.

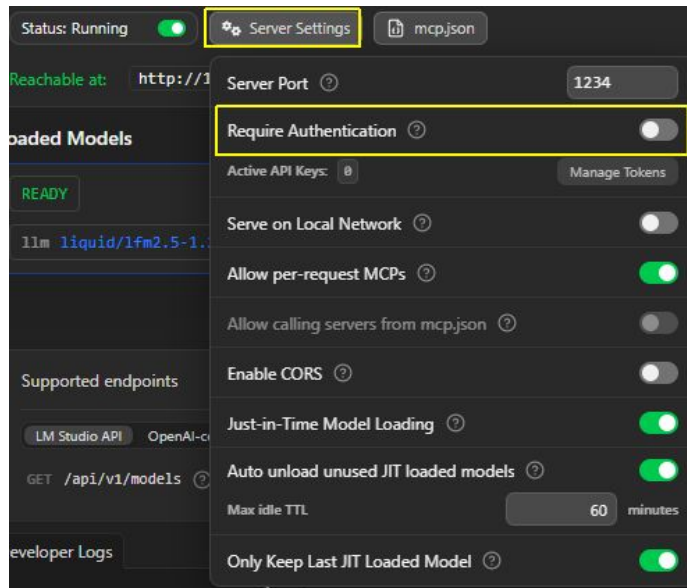
The main application will also determine which LLM models you can run, depending on the resources available on your system.

In our case, I recommend starting **X-Plane first, and only then the LLM server, if you want to use it.**

Starting the LM Studio web service

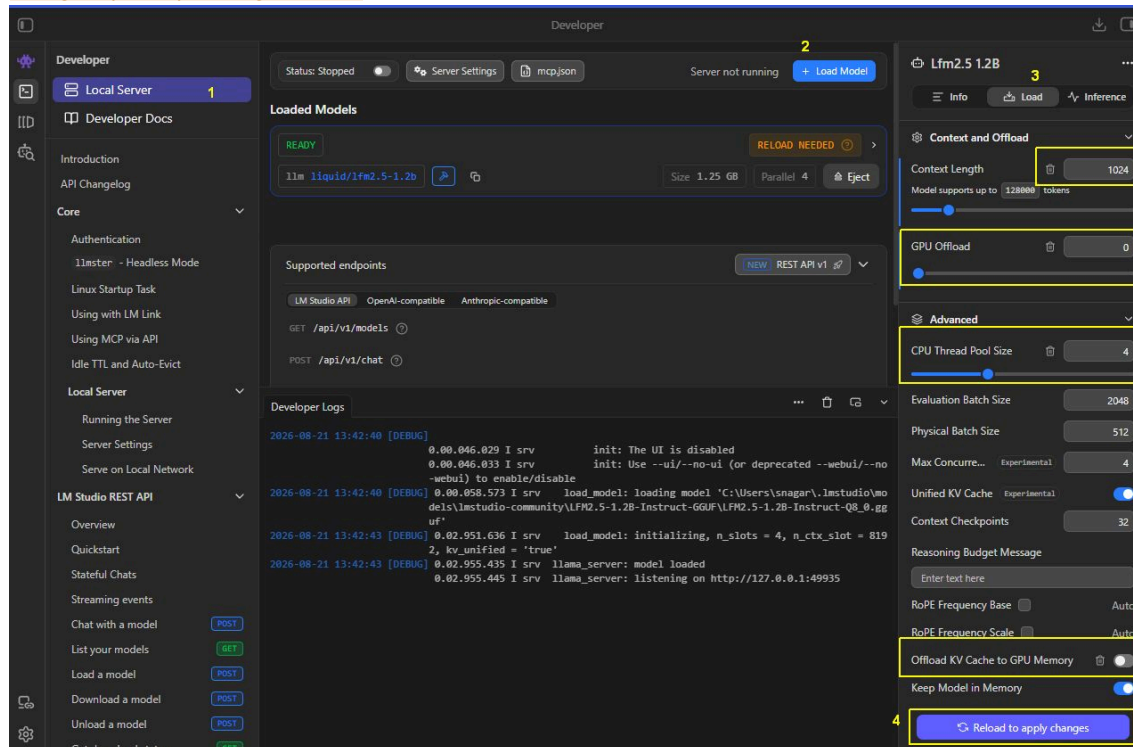
- Click the **Developer** tab (</>) on the left sidebar.
- Select your downloaded .gguf file from the **Select a model to load** dropdown at the top.
 - After loading the model, check the "load" tab on the right and consider modifying the settings, as suggested in previous topic.
You might need to press the **[Reload and apply changes]** button, so changes will take effect.
- In the Server Settings panel:
 - Port:** Default is 1234 (or change as needed).
 - CORS:** Enable if calling from web environments.
 - Require Authentication:** Ensure this is toggled **OFF** unless you want to generate a explicit Bearer key.
- Click the green **Start Server** button.

Before starting the server, check its settings:

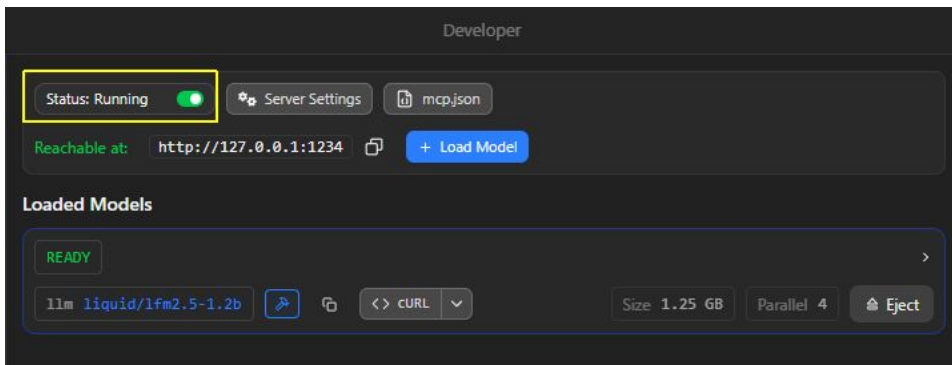


Load the model and tweak its parameters (if needed).

The demonstration here is intended to use only the CPU, instead of the GPU. You can increase the "GPU Offload" setting to speed up token generation.

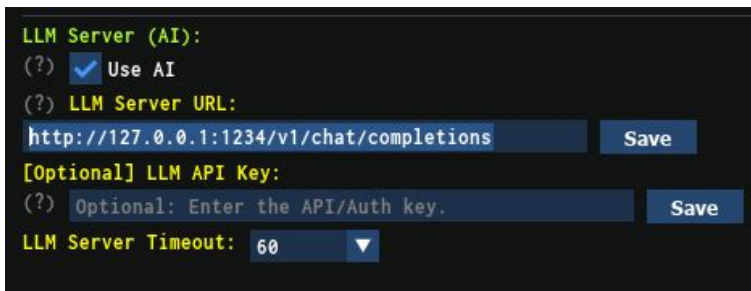


Start the Server:



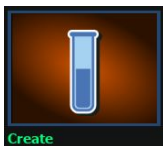
The order of these steps is unimportant, you can change the **LLM parameters** and **Reload** how many times you need, the web server is running independently from it.

Mission-X setup



Once the LLM server is up and running, you can add it in the Mission-X setup screen.

- Open the Setup screen and expend the "Simbrief & ... & LLM setup" header.
- Scroll to the LLM section.
- Check the "Use AI".
- Enter the LLM server URL. Make sure the virtual path adhere to the OpenAI format (red):
Example: <http://127.0.0.1:1234/v1/chat/completions>
- If you need API Key you could add it.



To use the LLM with Mission-X, generate a random mission from the **"Generate Random Mission"** screen.

- As of this writing, the LLM server is used only to generate the mission description.
- If the LLM server is unavailable, Mission-X will automatically generate the mission description without using the LLM.

Notes

- You can use any LLM server you prefer, such as **LM Studio**, **llama.cpp**, or **Ollama**.
- When providing the server URL, you must use the **OpenAI-compatible API endpoint**. The URL consists of two parts:
 - The **base URL** of the LLM server, for example:
`http://127.0.0.1:1234/`
 - The **API path** that is appended to the base URL, allowing the server to route the request to the appropriate API endpoint.
For example, with **LM Studio**, the API path is: `v1/chat/completions`
- You can experiment with different models, but I recommend using smaller models that are designed for text generation, or models that produce mission descriptions you particularly like.